PREDICTIVE MAINTENANCE
Final Project Report

Harshit Rathi | AIML Capstone Project 3

10 November 2026 | Model deployment and automation

Executive overview

This project tests whether six engine sensor readings can distinguish the two recorded condition classes, then makes the selected model available through a Streamlit app and a repeatable GitHub Actions workflow. The dataset contains 19,535 observations. A stratified holdout of 3,907 observations is reserved for final evaluation.

The committed random forest achieves 0.80 average precision, 0.69 ROC AUC and 0.65 balanced accuracy. The prior baseline scores 0.63, 0.50 and 0.50, respectively. The model finds useful associations, but substantial classification errors remain.

The public app and successful automation demonstrate that the model can be reproduced and served. The healthy/faulty label mapping and several measurement units remain unverified. Without engine identifiers, timestamps or future failure outcomes, this is condition classification rather than demonstrated advance failure prediction.

Submission files

Harshit_Rathi_PredictiveMaintenance_FinalReport.pdf

Harshit_Rathi_PredictiveMaintenance_Notebook.html

The PDF presents findings and business implications. The executed HTML notebook contains the full code, experiment logs and verification outputs. Raw implementation excerpts appear only in the appendices.

GitHub repository

Live Streamlit Community Cloud app

1. Data registration and background

Methodology

Load the CSV directly from the repository data folder, check all expected columns and numeric values, verify a binary target, count missing values and duplicates, and record a SHA-256 checksum. The same validation is used locally and in GitHub Actions.

Source: the supplied engine_data.csv matches Parv Modi's Automotive Vehicles Engine Health Dataset, Kaggle version 1; the earlier source verification was recorded on 2 November 2026 [1]. The dataset documentation does not establish the measurement location, collection protocol or whether rows represent repeated engines.

Table 1. Dataset registration summary

Check	Finding
Observations / columns	19,535 / 7
Predictors	Engine rpm, Lub oil pressure, Fuel pressure, Coolant pressure, lub oil temp, Coolant temp
Target	Engine Condition: original labels 0 and 1
Class 0	7,218 (36.95%)
Class 1	12,317 (63.05%)
Missing values / duplicate rows	0 / 0
Stored source	data/engine_data.csv; checksum recorded in Appendix A

The target meaning requires clarification. Nafar describes class 1 as healthy [2], while Chen describes class 1 as requiring maintenance [3]. These sources conflict; the original source metadata does not resolve the mapping. Class 1 is therefore positive only for statistical evaluation, not a confirmed failure label.

Business implication: obtain the data dictionary and collection history before using a prediction to schedule maintenance. Neither the file name nor differences in sensor means establish what either class means.

2. Exploratory data analysis

Methodology

Use only the 15,628 training observations. Summarize sensor distributions, compare classes and examine Pearson correlations. Flag observations beyond 1.5 interquartile ranges for review without deleting them.

Table 2. Training sensor overview

Sensor	Mean	Std. dev.	Median	Minimum	Maximum
Engine rpm	792.26	267.34	748.00	61.00	2239.00
Lub oil pressure	3.31	1.02	3.16	0.00	7.27
Fuel pressure	6.65	2.76	6.19	0.00	21.14
Coolant pressure	2.34	1.04	2.17	0.00	7.48
lub oil temp	77.65	3.12	76.82	71.32	89.58
Coolant temp	78.43	6.22	78.34	62.45	195.53

Only rpm has an explicit unit. Pressure and temperature values retain the dataset's original scale. Small positive pressures may round to 0.00; they are not missing values.

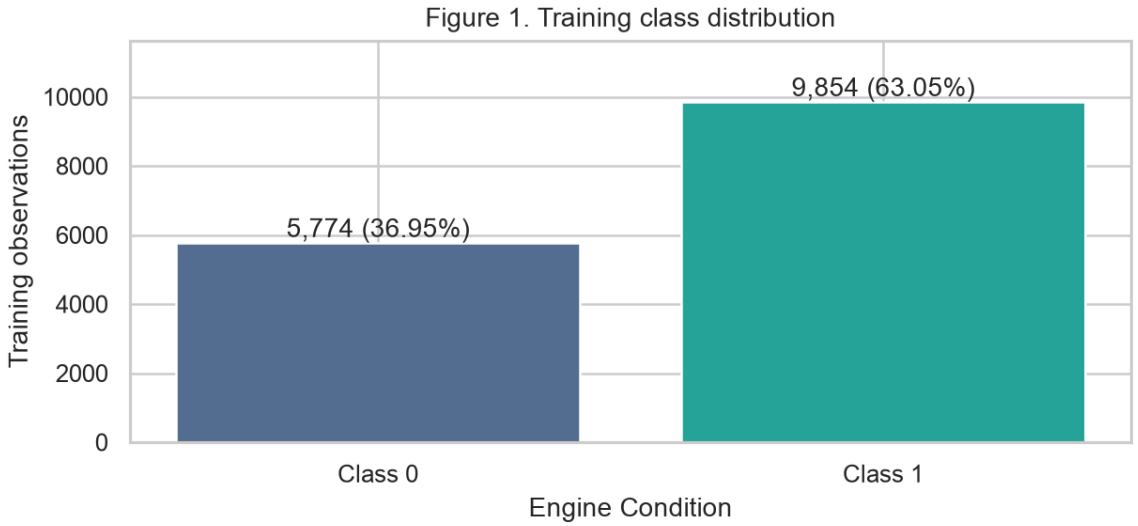


Figure 1. Training class distribution; class 1 is the majority.

Class 1 represents 63.05% of training observations. A majority-class prediction can therefore appear accurate while never recognizing class 0; balanced accuracy and both-class diagnostics are needed.

Business implication: evaluating only overall accuracy could hide failure to recognize the smaller class. Use both-class diagnostics and compare with a prior-probability baseline.

2.1 Univariate distributions

Figure 2. Sensor distributions (training data)

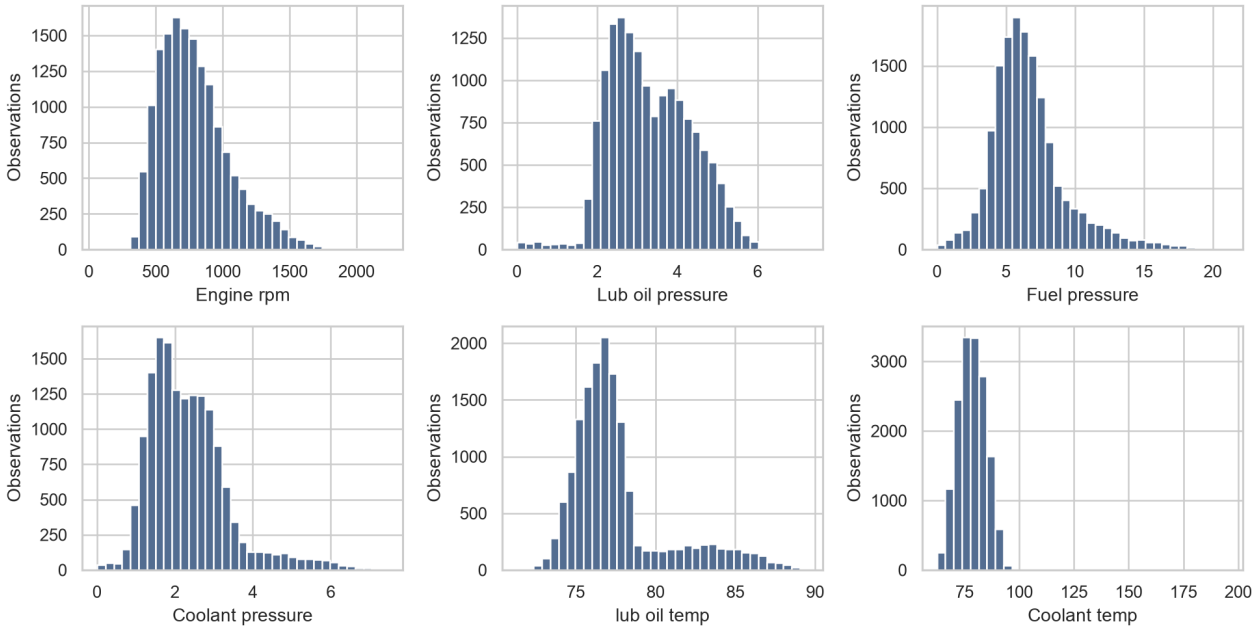


Figure 2. Training distributions retain their full observed ranges.

Engine speed and pressure measurements have upper tails. Oil temperature is concentrated near the mid-to-high 70s, with a smaller high-temperature tail. A few high coolant readings stretch the displayed range, so the median and spread help describe typical observations.

Coolant temp has a median of 78.34 and a maximum of 195.53; 2 readings lie outside its 1.5-IQR limits. Retain these readings because they may describe unusual operating conditions; verify units and sensor calibration before applying physical limits.

Table 3. Outlier review using the 1.5-IQR rule

Sensor	Flagged training observations	Treatment
Engine rpm	371	Retained pending domain validation
Lub oil pressure	52	Retained pending domain validation
Fuel pressure	909	Retained pending domain validation
Coolant pressure	641	Retained pending domain validation
lub oil temp	2107	Retained pending domain validation
Coolant temp	2	Retained pending domain validation

Business implication: unusual readings could be valid operating states or measurement problems. Review calibration and units before excluding them; automatic removal could hide relevant conditions.

2.2 Bivariate class comparisons

Figure 3. Sensor distributions by class (training data)

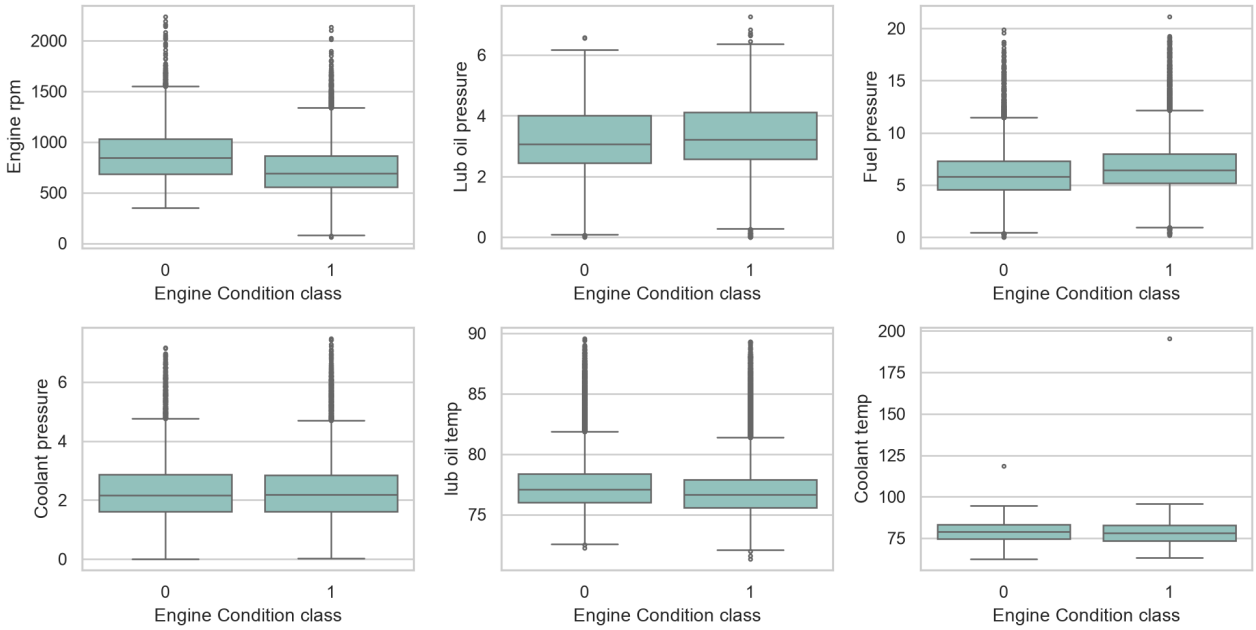


Figure 3. Sensor distributions overlap between the recorded classes.

Table 4. Mean sensor readings by class

Sensor	Class 0	Class 1	Difference: 1 – 0
Engine rpm	885.74	737.49	-148.24
Lub oil pressure	3.23	3.35	0.12
Fuel pressure	6.23	6.90	0.68
Coolant pressure	2.37	2.33	-0.04
lub oil temp	78.03	77.43	-0.60
Coolant temp	78.81	78.21	-0.60

Engine rpm averages 885.74 for class 0 and 737.49 for class 1. The distributions overlap, so rpm alone should not be used as a maintenance rule.

Class 1 also has higher mean fuel pressure and lower mean oil temperature in this sample. The overlap supports testing combinations of sensors rather than adopting a single cutoff. These relationships describe association, not causal effects or a healthy/faulty interpretation.

2.3 Multivariate analysis

Figure 4. Multivariate Pearson correlations (training data)

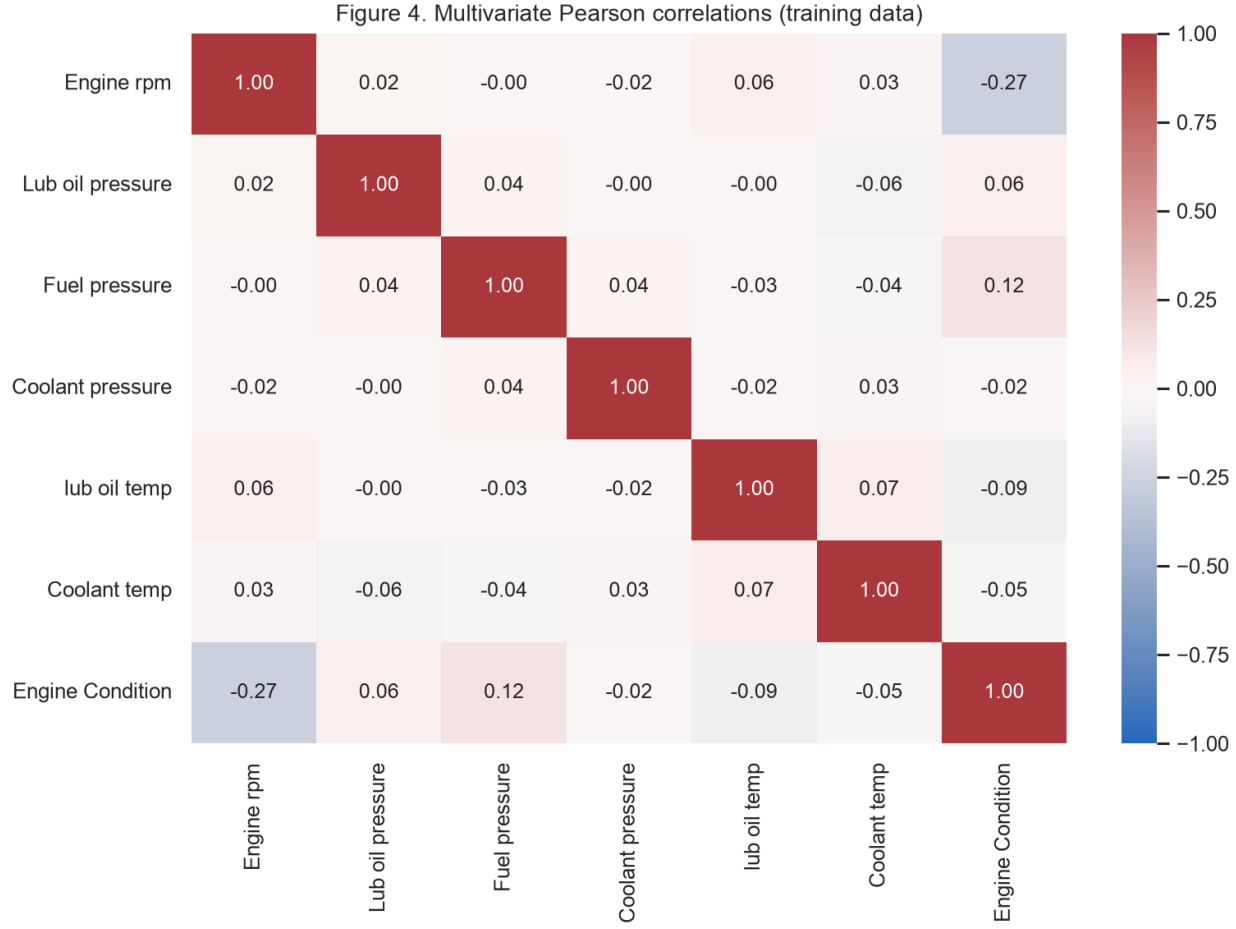


Figure 4. Pearson correlations across the training sensors and encoded target.

Engine rpm has the strongest absolute linear association with the target (r = -0.27). This describes an association with the encoded class, not a cause of engine condition.

The strongest sensor-to-sensor correlation is lub oil temp versus Coolant temp (r = 0.07). Weak pairwise correlations do not rule out nonlinear relationships or interactions.

Business implication: no strong linear redundancy justifies dropping a sensor on correlation grounds alone. A random forest can test nonlinear combinations while retaining all available measurement context. The absence of strong linear correlations does not establish that sensors are independent.

3. Data preparation

Methodology

Retain the six sensors, reject conflicting labels for identical predictor rows, and remove exact duplicates if present. Save a stratified 80/20 split with seed 42. Learn median imputation within each training fold; the test data are excluded from fitting and parameter selection.

Table 5. Preparation decisions

Step	Decision and reason
Source loading	Read data/engine_data.csv directly from the repository.
Column removal	None: no identifier or explicitly outcome-derived field is present. All six sensors are retained.
Cleaning	No missing values or duplicate rows observed; no rows removed. Retain extreme values pending domain checks.
Feature engineering	Use the original six readings. No unsupported unit-dependent ratios or time features are invented.
Scaling	Not required for the tree-based model.
Train / test split	15,628 / 3,907 rows, stratified by class; seed 42.
Artifact handoff	Preparation saves train/test CSVs and metadata, then uploads the split artifact. Training downloads it and validates checksums.
Leakage controls	Target excluded from predictors; preprocessing fitted within folds; held-out data excluded from tuning.

The saved split checksums agree between the notebook and the hosted workflow. This connects the report's evaluation to the data used to produce the committed model; detailed values appear in Appendix A and the HTML notebook.

Limitation: random splitting assumes sufficiently independent observations. The file lacks engine identifiers and timestamps, so repeated-engine leakage and performance on later operating periods cannot be measured. A future operational evaluation should separate engines and time periods.

4. Model building and experiment tracking

Methodology

Fit a random forest and compare it with a prior-probability dummy classifier. Search 16 parameter combinations using three shuffled stratified folds, producing 48 cross-validation fits. Select the highest mean average precision for class 1, refit the selected pipeline, and save it with all experiment records.

Table 6. Hyperparameter search

Parameter	Candidate values
Number of trees	100, 200
Maximum depth	Unrestricted, 10
Minimum samples per leaf	1, 3
Class weights	None, balanced
Fixed settings	Seed 42; three stratified folds; average precision for model selection

Selected model: 100 trees, maximum depth 10, minimum 1 sample(s) per leaf and balanced class weighting. Mean validation average precision is 0.79, with fold standard deviation 0.01; mean training average precision is 0.93.

The higher training score indicates remaining overfitting. Close candidate scores do not establish that the selected model will always be best on a different sample. Depth restriction helps control complexity without assuming a linear sensor relationship.

Average precision measures how well class-1 observations are ranked across thresholds. The baseline is the class-1 prevalence. Selection by this metric does not guarantee the best F1 score or the lowest maintenance cost at the default threshold.

All candidate parameters, fold scores, ranks and timings are retained. The saved model contains both preprocessing and the fitted forest. The report’s performance figures refer to the committed hosted model used by Streamlit; the notebook also includes a separate fresh local reproduction.

Full tuning results are in Appendix B and the executed HTML notebook. Raw code and execution logs are kept outside the report’s main narrative.

4.1 Held-out model performance

Table 7. Performance on 3,907 held-out observations

Metric	Prior baseline	Committed forest
Average Precision	0.63	0.80
ROC AUC	0.50	0.69
Balanced Accuracy	0.50	0.65
Precision	0.63	0.76
Recall	1.00	0.63
F1	0.77	0.69

Figure 5. Held-out ranking performance (positive class = 1)

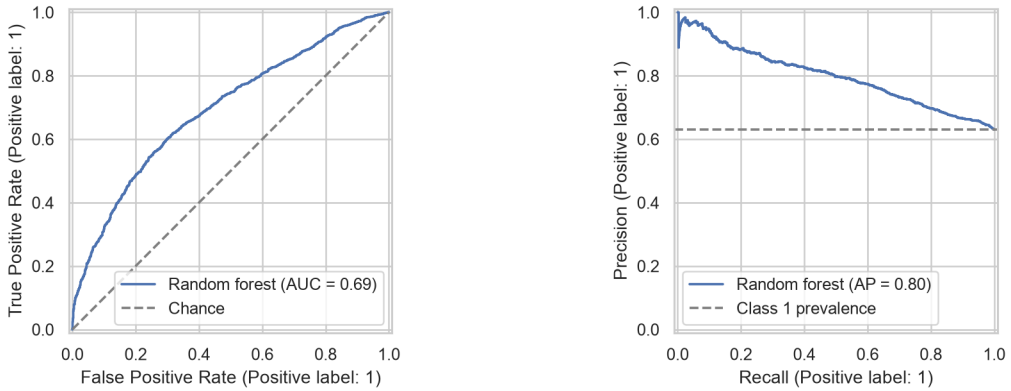


Figure 5. ROC and precision-recall curves for the committed model; positive class = 1.

Average precision improves by 0.17 over the baseline. Balanced accuracy averages recall for both classes, so the improvement above 0.50 reflects recognition of both labels rather than majority-class guessing. ROC AUC measures ranking discrimination.

Class-1 F1 at threshold 0.50 is 0.69, below the baseline's 0.77. The baseline predicts class 1 for every observation, giving perfect class-1 recall but no class-0 recognition. The forest trades some class-1 recall for class-0 recognition. This tradeoff must be assessed against confirmed business costs.

4.2 Errors and their interpretation

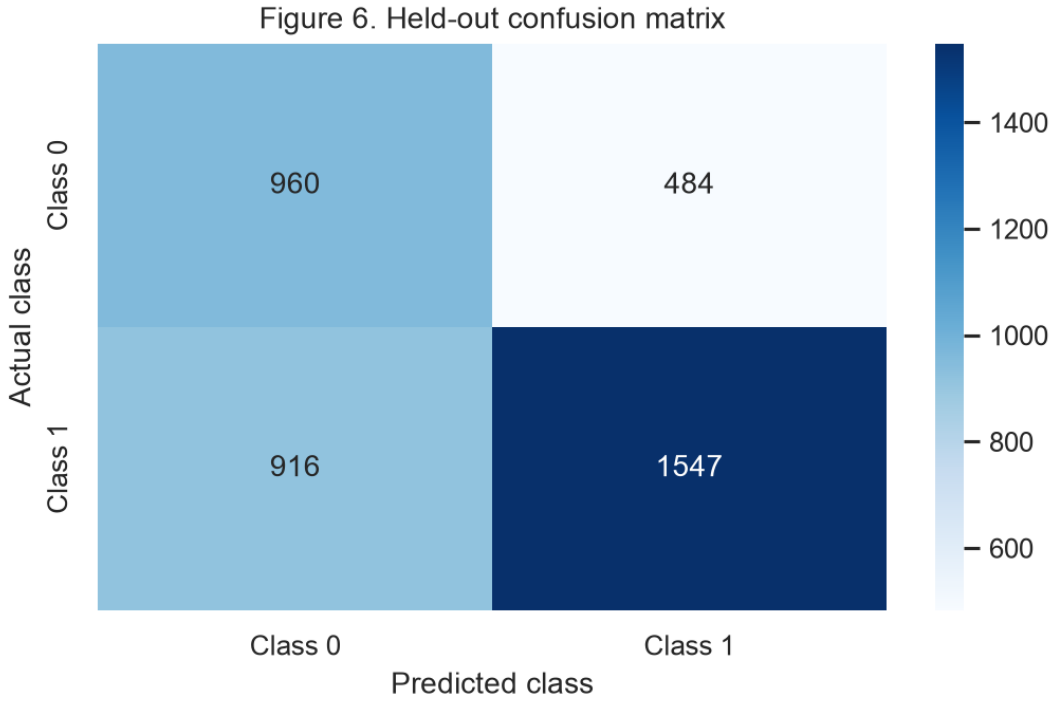


Figure 6. Confusion matrix on the held-out set. Rows are actual labels; columns are predictions.

Table 8. Classification outcomes

Actual class	Correct predictions	Incorrect predictions	Recall
Class 0	960	484	0.66
Class 1	1,547	916	0.63

The model predicts class 1 for 2,031 of 3,907 observations (51.98%). These are dataset-class predictions, not confirmed maintenance alerts.

Neither error type can be called a missed fault or false alarm until label semantics are verified. If class 1 is later confirmed as the intervention class, its missed observations would require particular attention; if class 0 is the intervention class, the interpretation reverses. No label mapping is assumed here.

The decision threshold remains at the model default. A future threshold study should use validation data and an agreed cost for each error type, while preserving a separate test set for the final assessment.

5. Model deployment

Methodology

Deploy the Streamlit entry point from the repository's main branch on Community Cloud using Python 3.12 and the dependencies file [4]. Load the committed model once, collect six readings into a dataframe in the training feature order, and show the predicted class and class-1 model score.

Table 9. Deployment implementation

Requirement	Implementation
Repository model	The app loads models/best_model.joblib, including its fitted preprocessing.
Input capture	Six numeric inputs are stored in a one-row dataframe with the original sensor column names.
Input checks	Reject non-finite values; warn when values are outside observed training ranges.
Prediction	Display class 0/1 and the uncalibrated class-1 score; allow a CSV download of the assessment.
Dependencies	requirements.txt pins the core training/inference packages to matching versions.
Hosting	Streamlit Community Cloud; main branch; streamlit_app.py; Python 3.12.
Model integrity	Validate the model checksum and feature schema before inference; use the repository model only.

Matching training and serving dependencies reduces model-loading incompatibility [5]. The model is loaded from versioned files and is not retrained for each visitor. Inputs remain in the current session unless the user downloads the assessment; this demonstration does not create a prediction log.

Using the displayed training-median defaults, the app predicts class 1 with a class-1 score of 57.19%. This is a reproducible functional check, not a measured probability of failure.

Open the deployed app

Business implication: the interface makes the model available for analyst review without requiring a notebook. Operational decisions still require verified class definitions and validation on the intended fleet.

6. Automated GitHub Actions workflow

Methodology

Use separate preparation, training and model-persistence jobs in pipeline.yml. Transfer data and models as workflow artifacts [6], require upstream jobs to succeed, and commit the verified model bundle back to main using the workflow's write permission.

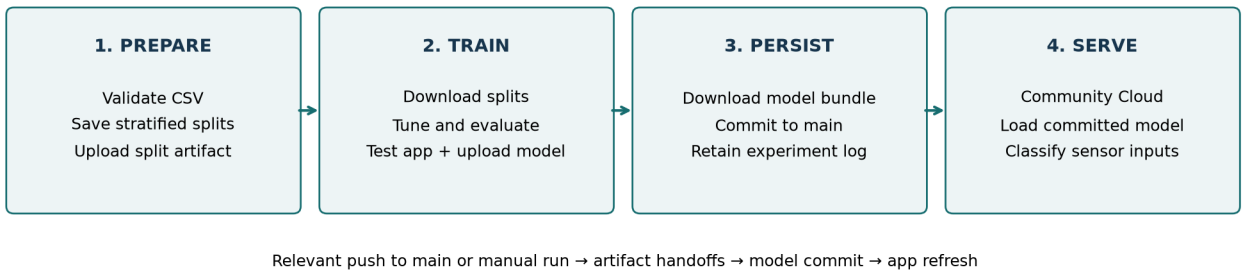


Figure 7. Automated training and deployment flow. Community Cloud follows the main branch after initial setup.

Table 10. Workflow stages and controls

Stage	Action / output
Trigger	Relevant source/data/dependency changes pushed to main, or manual workflow dispatch.
Prepare	Install dependencies, test pipeline behavior, validate data, save and upload train/test splits.
Train	Download splits, check hashes, tune model, evaluate holdout, save parameters, logs and provenance.
App verification	Run Streamlit and inference tests against the newly trained model before persistence.
Persist model	Download model artifact; commit changed model files and experiment records to main.
Serve	After initial deployment, Community Cloud follows the selected branch and loads the committed model.

Model-only commits do not match the workflow's source path filters, avoiding a training loop. Only the persistence job has repository write permission. Artifacts are retained for 90 days; the committed model, metrics and provenance remain in the repository.

Business implication: checks and explicit artifact handoffs reduce manual handling errors and connect each model to its input data and recorded experiment results. Automation does not replace model-quality review.

7. Output evaluation: repository evidence

Methodology

Inspect the published repository and capture its folder structure. Verify that the data, model, workflow, dependency file and app entry point are accessible from the submitted repository link.

GitHub repository: harshitrathi14/capstone-predictivemaintenance

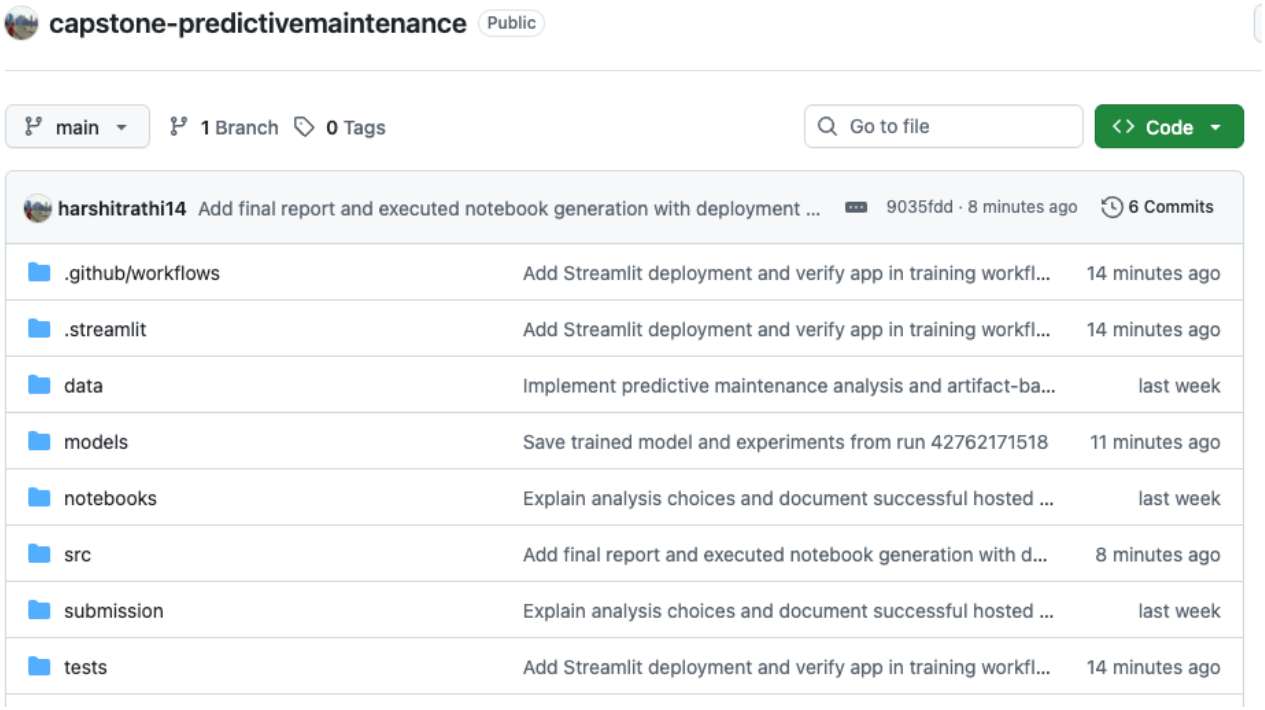


Figure 8. Actual GitHub repository, cropped to show its folder structure. The full capture is in the HTML notebook.

Table 11. Repository evidence map

Location	Purpose
data/	Registered engine dataset
src/	Validation, preparation, training and inference code
models/	Committed model, input profile, metrics, tuning log and workflow provenance
.github/workflows/pipeline.yml	End-to-end automation and automatic model commit
streamlit_app.py / requirements.txt	Deployment entry point and dependencies

7.1 Output evaluation: executed workflow

Successful workflow run 42762171518

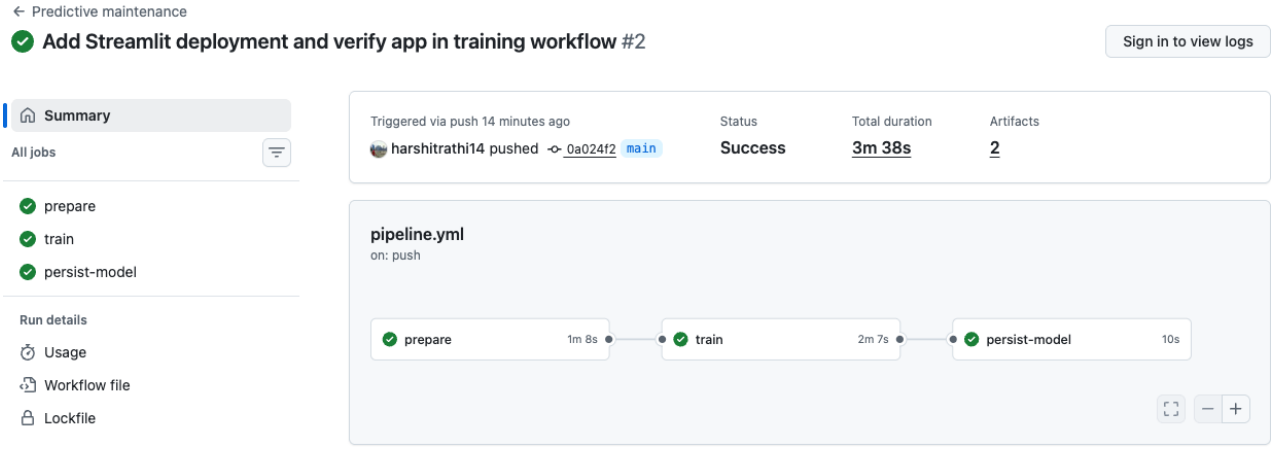


Figure 9. Actual GitHub Actions run summary and successful jobs. The full capture is in the HTML notebook.

Table 12. Hosted workflow verification

Check	Recorded result
Overall result	success
prepare	success
train	success
persist-model	success
Model persistence	Automatic model commit 5cef7f634a47 on main
Artifact handoff	Train/test splits passed to training; model and experiment bundle passed to persistence

The notebook independently checks that the held-out split hashes and recomputed committed-model metrics agree with the hosted records. This confirms that the evaluated model and the app's model belong to the same recorded training workflow. The local reproduction is shown separately in the HTML notebook.

7.2 Output evaluation: public Streamlit app

https://appapppy-mh87owifffvywmgi8dxfo8.streamlit.app/

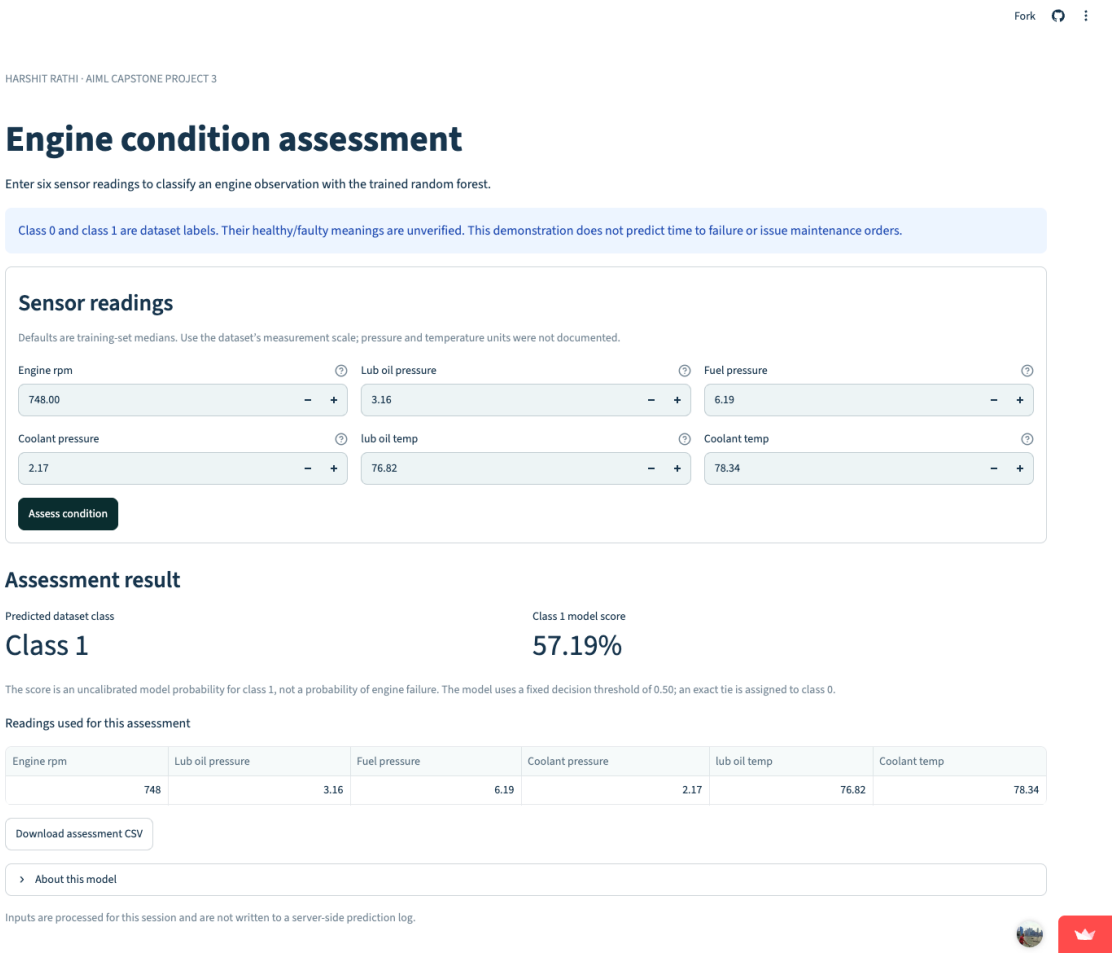


Figure 10. Actual public Streamlit app after submitting the default sensor readings.

The browser check submitted all six displayed defaults and returned class 1 with a class-1 score of 57.19%, matching the committed model's local inference. The public app was accessible when evidence was captured.

Seven automated checks cover invalid schema/labels, altered split artifacts, disjoint splits and model serialization, inference column order, incomplete/non-finite readings, and a Streamlit submission with an out-of-range warning. The fresh hosted model also passed the app checks before it was committed.

This evidence establishes functional deployment and reproducible inference. It does not establish production reliability, prospective failure detection or a measured reduction in maintenance cost.

Evidence captured: 10 November 2026, 11:11 AM IST.

8. Actionable insights and recommendations

Table 13. Recommendations tied to findings

Finding	Recommended action	Owner / decision
Label meanings and several units are unverified.	Obtain the authoritative data dictionary and confirm how labels were assigned.	Data owner + maintenance lead; prerequisite to operational interpretation.
Sensor ranges include unusual readings.	Review calibration and original measurements; do not remove extremes solely on a statistical rule.	Sensor/domain specialist; confirm data-quality rules.
The forest improves ranking but still makes substantial errors.	Choose an inspection threshold on validation data with agreed missed-condition and unnecessary-inspection costs.	Maintenance lead + data scientist; proceed only after label confirmation.
No engine IDs, timestamps or future outcomes are available.	Collect these fields and test on unseen engines and later periods.	Data engineering + operations; validate generalization and warning time.
The app and workflow make inference repeatable.	Use an analyst-reviewed pilot; monitor input ranges, class mix and later verified outcomes. Retain model versions for rollback.	ML owner; expand only when pilot results justify it.

Business value

The immediate value is a reproducible analytical process and an accessible demonstration. These reduce the effort needed to review sensor observations and repeat model experiments. No financial saving is quantified because failure costs, intervention effectiveness and confirmed outcomes are unavailable.

Decision boundary

Keep the current system as a supervised demonstration. Automatic maintenance orders, remaining-useful-life estimates and production claims would exceed the evidence supplied by this dataset. A prospective pilot with verified labels is the next step before operational adoption.

References

[1] Original dataset: Parv Modi, Automotive Vehicles Engine Health Dataset, Kaggle, version 1 (2023).

https://www.kaggle.com/datasets/parvmodi/automotive-vehicles-engine-health-dataset

The downloaded CSV exactly matches the supplied file. The retrieved source metadata does not define the target-class meanings.

[2] Hossein Nafar, master thesis, University of Wuppertal (2024), printed page 30 / PDF page 41.

https://www.vzu.uni-wuppertal.de/fileadmin/site/vzu/Masterarbeit_HosseinNafar.pdf#page=41

Describes class 1 as healthy and class 0 as out of operation or failed; reference 47 cites the matching Kaggle dataset.

[3] Jayden P. Chen, Fusion of Neural Networks and Logistic Regression for Predictive Maintenance of Vehicle Engines (2024), Table 1, page 66.

https://ajosr.org/wp-content/uploads/journal/published_paper/volume-2/issue-2/ajsr2024_pFcUxi32.pdf#page=3

Describes class 1 as indicating a need for maintenance. Its sensor summary statistics match this dataset, although its stated row count differs.

[4] Streamlit documentation: Deploy your app on Community Cloud. Accessed 10 November 2026.

https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy

[5] scikit-learn documentation: Model persistence. Accessed 10 November 2026.

https://scikit-learn.org/stable/model_persistence.html

[6] GitHub documentation: Store and share data with workflow artifacts. Accessed 10 November 2026.

https://docs.github.com/en/actions/tutorials/store-and-share-data

The dataset is externally sourced and attributed above. Analysis, code, figures and interpretation were prepared for this project. Evidence screenshots show the project's actual GitHub repository, hosted workflow and public app; no external stock images are used.

Appendix A. Reproducibility and evidence

The main report uses the committed hosted model. The final notebook re-executes the complete analytical pipeline locally, then independently evaluates that hosted model and verifies its recorded metrics. Different platforms may produce slightly different floating-point tree splits; local results are not substituted for the hosted model's results.

Table A1. Hosted training environment

Component	Version / value
Python	3.12.16
pandas	2.3.3
numpy	2.5.4
scikit-learn	1.9.1
joblib	1.6.0
Seed / folds / candidates	42 / 3 / 16
Inference framework	Streamlit 1.67.0
Source commit	0a024f274d62
Model commit	5cef7f634a47

Source CSV SHA-256

616c7b985ec00e299061f02b11fd866f8eca16b5e8d15d4a61a5fbfd116b8045

train SHA-256

cb2ed6e952ec52c2b14d60165ba8bf9e77537e7f4c29215fe51a8712df659f78

test SHA-256

25c2dc140bd54e619db312d40906ca20cfa711027814e435bdfc74a95cd5472f

Committed model SHA-256

00927ad943f5eec6ab496ab0c83614c1425c579dff1853f7430732dbdd076068

Verification record

Captured URLs, timestamps, workflow jobs and test results are displayed in notebook section 7. Full registration, split metadata, environments and model parameters are printed in notebook sections 1–5. Full code and outputs are included there to keep this report readable.

Appendix B. Extended experiment results

Table B1 reports all 16 hosted candidates. Extra decimal precision is retained here to distinguish close scores. Full fold scores, timings and training scores are displayed in notebook section 4 and saved in models/experiments.csv. The notebook also includes the separate fresh local tuning run.

Table B1. Complete hosted cross-validation comparison

Trees	Depth	Leaf	Weight	CV AP	CV std.	Rank
100	None	1	None	0.77091	0.00467	16
200	None	1	None	0.77430	0.00497	14
100	None	3	None	0.78067	0.00453	12
200	None	3	None	0.78267	0.00588	10
100	10	1	None	0.79257	0.00541	5
200	10	1	None	0.79248	0.00548	6
100	10	3	None	0.79152	0.00498	8
200	10	3	None	0.79275	0.00517	4
100	None	1	balanced	0.77374	0.00425	15
200	None	1	balanced	0.77710	0.00473	13
100	None	3	balanced	0.78127	0.00594	11
200	None	3	balanced	0.78320	0.00662	9
100	10	1	balanced	0.79383	0.00507	1
200	10	1	balanced	0.79367	0.00551	2
100	10	3	balanced	0.79238	0.00447	7
200	10	3	balanced	0.79313	0.00523	3

AP = average precision for class 1; CV = cross-validation. Rank is based on mean validation AP. The held-out test set is not used to rank candidates. The saved best_parameters.json also records all fixed estimator and preprocessing parameters.

Appendix C. Implementation excerpts

These excerpts document the deployed inference and artifact handoff. The complete analytical implementation is executed in the HTML notebook; complete app source and workflow YAML are displayed in sections 5 and 6.

C1. Validate inputs and classify an observation

```
def predict_observation(values, model, profile):
    features = profile['features']
    if set(values) != set(features):
        raise ValueError('Provide exactly the six required sensor readings.')
    frame = pd.DataFrame([values], columns=features).astype(float)
    if not np.isfinite(frame.to_numpy()).all():
        raise ValueError('Every sensor reading must be a finite number.')
    outside = [name for name in features
               if not profile['summary'][name]['minimum'] <= frame.iloc[0][name]
               <= profile['summary'][name]['maximum']]
    prediction = int(model.predict(frame)[0])
    positive_index = list(model.classes_).index(1)
    score = float(model.predict_proba(frame)[0, positive_index])
    return frame, prediction, score, outside
```

C2. Pass prepared data between workflow jobs

```
# Preparation job
- uses: actions/upload-artifact@v4
  with:
    name: train-test-splits
    path: artifacts/splits/
# Training job
- uses: actions/download-artifact@v4
  with:
    name: train-test-splits
    path: artifacts/splits/
- name: Tune and evaluate using downloaded split artifacts
  run: python src/pipeline.py train
```

C3. Model persistence

After app verification, the persist-model job downloads the model bundle, stages models/, commits changed files with the workflow run ID and pushes to main. Its exact executable shell steps and successful execution record appear in the HTML notebook. No raw logs are embedded in the business narrative.